Databricks — Complete Handbook

lakehouse platform, compute, Unity Catalog, Delta Lake, Auto Loader, Lakeflow pipelines and jobs, performance, security and CI/CD

notes by Sagar · Jan 2026

Contents

1. What Databricks is
2. Workspace objects
3. Compute
4. Notebooks and dbutils
5. Storage and data access
6. Unity Catalog
7. Delta Lake
8. Medallion architecture
9. Ingestion
10. Lakeflow Declarative Pipelines (formerly Delta Live Tables)
11. Lakeflow Jobs (orchestration)
12. Databricks SQL
13. Performance on Databricks
14. Security
15. CI/CD and DevOps
16. Cost management
17. Monitoring and troubleshooting
18. End-to-end example

Sagar

1. What Databricks is

Databricks is a managed data and AI platform built around Apache Spark and Delta Lake. It runs on Azure, AWS and GCP. You get managed Spark clusters, a SQL engine, notebooks, orchestration, governance (Unity Catalog) and ML tools in one workspace, reading and writing data in your own cloud storage.

The lakehouse idea

	Data lake	Data warehouse	Lakehouse
Storage	Cheap object storage, open files	Proprietary storage	Cheap object storage, open table formats
Data types	Any (structured, semi, unstructured)	Structured	Any
ACID transactions	No	Yes	Yes (Delta / Iceberg)
Performance for BI	Poor without extra work	Excellent	Good to excellent
Workloads	Data science, raw storage	BI, reporting	Both on one copy of data

The lakehouse keeps data in open formats on cloud storage, adds a transaction layer (Delta Lake) for reliability, and adds governance and fast engines on top.

Control plane and compute plane

- **Control plane** (managed by Databricks): the web app, notebooks, job scheduler, cluster manager, Unity Catalog metadata.
- **Compute plane:** where clusters actually run and process data.
- *Classic compute* runs in **your** cloud account (VNet/VPC).
- *Serverless compute* runs in Databricks' account, starts in seconds, and is managed for you.
- **Data** stays in your cloud storage (ADLS Gen2, S3, GCS).

Account and workspaces

- An **account** holds users, groups, Unity Catalog metastores and billing.
- A **workspace** is an environment where teams build: notebooks, jobs, clusters. Companies usually have workspaces per environment (dev / test / prod) or per business unit, all attached to the same Unity Catalog metastore for a region.

2. Workspace objects

Object	What it is
Notebook	Interactive document with cells in Python, SQL, Scala or R
Git folder (Repos)	A Git repository cloned into the workspace — the way to version code
Workspace files	Python modules, configs and other files stored in the workspace
Catalog Explorer	UI to browse catalogs, schemas, tables, volumes, lineage and permissions
Jobs	Scheduled/triggered workflows (Lakeflow Jobs)
Pipelines	Declarative ETL pipelines (Lakeflow Declarative Pipelines, formerly Delta Live Tables)
SQL editor, queries, dashboards, alerts	Databricks SQL objects
Compute	Clusters, SQL warehouses, pools

Development flow

1. Develop in notebooks or Python files inside a Git folder.
2. Commit and push to a branch; raise a pull request.
3. CI runs tests and deploys to test, then prod, using Databricks Asset Bundles or the CLI/API.
4. Production runs as jobs, not as manually executed notebooks.

Sagar

3. Compute

Types of compute

Compute	Purpose	Billing pattern
All-purpose cluster	Interactive development, shared by users	More expensive per DBU
Job cluster	Created for one job run, terminated after	Cheaper per DBU; isolated per run
Serverless compute (notebooks, jobs, pipelines)	Instant start, no cluster management	Pay per use; managed by Databricks
SQL warehouse (classic / pro / serverless)	SQL queries, BI tools, dashboards	Per warehouse uptime
Instance pool	Keeps idle VMs ready to cut cluster start time	VM cost while idle

> Production pipelines should run on **job clusters or serverless jobs**, not on all-purpose clusters. They're cheaper, isolated, and start clean every run.

Cluster configuration choices

- **Databricks Runtime (DBR):** a bundle of Spark version, Delta, libraries and optimizations. Use an **LTS** (long-term support) version in production. ML runtimes add ML libraries.
- **Photon:** Databricks' vectorized C++ engine for SQL and DataFrame operations. Big speedups for scans, joins, aggregations; higher DBU rate. Not used for Python UDFs or RDD code.
- **Node types:** memory-optimized for joins/caching, compute-optimized for CPU-heavy work, storage-optimized (local SSD) for disk cache and heavy shuffles.
- **Autoscaling:** min and max workers; Databricks adds workers when tasks queue up.
- **Auto-termination:** stop idle all-purpose clusters after N minutes.
- **Spot instances:** cheaper VMs that the cloud can reclaim — good for workers on fault-tolerant batch jobs; keep the driver on-demand.
- **Single-node clusters:** driver only — for small data, pandas, testing.

Access modes (Unity Catalog)

Mode	Who	Notes
Standard (formerly shared)	Multiple users, isolated from each other	Recommended default; some low-level APIs restricted
Dedicated (formerly single user)	One user or group	Full API support (RDD, some ML libraries)
No isolation (legacy)	Anyone	No Unity Catalog — avoid

Cluster policies

Admins define policies that limit what users can create: allowed node types, max workers, mandatory tags, required auto-termination. Policies are the main way to control cost and enforce standards.

Libraries and init scripts

- Install Python wheels, PyPI packages or Maven JARs at cluster, notebook (**%pip install**) or job-task level.
- Init scripts run on each node at startup (install OS packages, set environment). Store them in workspace files or volumes.
- Prefer pinned versions and a private package index for reproducibility.

Sagar

4. Notebooks and dbutils

Magic commands

```
%python  %sql  %scala  %r     switch language for a cell
%md                          markdown documentation
%sh                          shell command on the driver
%fs ls /Volumes/main/raw/landing        file system commands
%pip install pkg==1.2.3      notebook-scoped Python library
%run ./utils/common          run another notebook inline (shares variables)
```

dbutils

```
dbutils.fs.ls("/Volumes/main/raw/landing/")
dbutils.fs.cp(src, dst, recurse=True)
dbutils.fs.rm(path, recurse=True)
dbutils.fs.mkdirs(path)

dbutils.secrets.get(scope="kv-prod", key="sql-password")      # value is redacted in output
dbutils.secrets.listScopes()

dbutils.widgets.text("run_date", "2026-01-15")
run_date = dbutils.widgets.get("run_date")

result = dbutils.notebook.run("./child", timeout_seconds=600, arguments={"run_date": run_date})
dbutils.notebook.exit("ok")                                   # return a value to the caller

dbutils.jobs.taskValues.set(key="row_count", value=12345)     # pass values between job tasks
dbutils.jobs.taskValues.get(taskKey="extract", key="row_count", default=0)
```

%run vs dbutils.notebook.run

	%run	dbutils.notebook.run
Runs in	Same context	Separate ephemeral job
Shares variables/functions	Yes	No — only string arguments in, one string out
Use for	Including shared helper code	Orchestrating notebooks with parameters, parallel runs

For production, prefer Python modules (imported normally) and job tasks over notebook chaining.

display

display(df) renders a table with built-in charts and profiling. It runs a limited query (first rows), so it doesn't show totals — use explicit aggregations to check counts.

5. Storage and data access

Paths you'll see

Path	Meaning
`abfss://container@account.dfs.core.windows.net/path`	ADLS Gen2 (Azure)
`s3://bucket/path`	Amazon S3
`gs://bucket/path`	Google Cloud Storage
`/Volumes/catalog/schema/volume/path`	Unity Catalog volume (governed files)
`dbfs:/...`	Databricks File System — legacy workspace storage
`/mnt/...`	DBFS mounts — legacy, replaced by external locations and volumes

Unity Catalog way of accessing storage

1. **Storage credential:** wraps cloud identity — on Azure, an Azure Databricks access connector (managed identity) with RBAC on the storage account.
2. **External location:** a storage path + a storage credential. Grant **READ FILES** / **WRITE FILES** / **CREATE EXTERNAL TABLE** on it.
3. **Volumes:** governed folders for non-tabular files (CSV drops, images, JARs). Managed or external.
4. **Tables:** managed or external, governed by Unity Catalog.

```
CREATE STORAGE CREDENTIAL IF NOT EXISTS lake_cred ...;      -- usually created by admins in UI/Terraform
CREATE EXTERNAL LOCATION raw_landing
  URL 'abfss://raw@mylake.dfs.core.windows.net/landing'
  WITH (STORAGE CREDENTIAL lake_cred);
GRANT READ FILES ON EXTERNAL LOCATION raw_landing TO `data-engineers`;

CREATE VOLUME main.raw.landing;                             -- managed volume
```

Avoid mounts and account keys in notebooks. With Unity Catalog, access is granted to identities, audited, and nobody handles storage keys.

6. Unity Catalog

Unity Catalog (UC) is Databricks' governance layer: one place to manage data objects, permissions, lineage and auditing across workspaces.

Object hierarchy

```
Metastore (one per region, attached to workspaces)
└── Catalog            e.g. dev, prod, or by domain: sales, finance
    └── Schema         (database) e.g. bronze, silver, gold
        ├── Table      managed or external
        ├── View / materialized view
        ├── Volume     files
        ├── Function   SQL/Python UDFs
        └── Model      ML models
```

Three-level names everywhere: **catalog.schema.table** — for example **prod.silver.orders**.

Managed vs external tables

	Managed	External
Data location	UC-managed storage location	A path you specify (in an external location)
DROP TABLE	Deletes data (recoverable for a short period)	Leaves the files
Maintenance	Eligible for predictive optimization	You run OPTIMIZE/VACUUM yourself
Recommended	Default for new tables	When other tools must own the files or paths are fixed

Permissions

```
GRANT USE CATALOG ON CATALOG prod TO `analysts`;
GRANT USE SCHEMA ON SCHEMA prod.gold TO `analysts`;
GRANT SELECT ON TABLE prod.gold.daily_sales TO `analysts`;
GRANT MODIFY ON SCHEMA prod.silver TO `etl-service-principal`;
GRANT CREATE TABLE ON SCHEMA dev.sandbox TO `data-engineers`;
REVOKE SELECT ON TABLE prod.gold.customers FROM `contractors`;
SHOW GRANTS ON TABLE prod.gold.daily_sales;
```

- Privileges are inherited downward: **SELECT** on a schema applies to all its tables.
- To read a table you need **USE CATALOG** + **USE SCHEMA** + **SELECT**.
- Grant to **groups**, not individuals. Run production jobs as **service principals**.
- Every object has an **owner** who can grant on it.

Fine-grained security

```
-- row filter: users see only their region
CREATE FUNCTION prod.sec.region_filter(region STRING)
RETURN is_account_group_member('admins') OR region = current_user_region();   -- illustrative logic
ALTER TABLE prod.gold.sales SET ROW FILTER prod.sec.region_filter ON (region);

-- column mask: hide email except for a group
CREATE FUNCTION prod.sec.mask_email(email STRING)
RETURN CASE WHEN is_account_group_member('pii-readers') THEN email ELSE '***' END;
ALTER TABLE prod.silver.customers ALTER COLUMN email SET MASK prod.sec.mask_email;
```

Dynamic views (views using **current_user() / is_account_group_member()**) are the older way to do the same.

Lineage, audit and discovery

- **Lineage:** UC captures table- and column-level lineage automatically for queries run on UC-enabled compute — which jobs and notebooks read and write each table.
- **Audit logs:** who accessed what, available in system tables.
- **Tags and comments:** classify sensitive columns (**pii**), document tables for discovery.
- **System tables** (**system.***): billing usage, audit, lineage, job runs, query history — query them with SQL for cost and operations reporting.

Delta Sharing

Open protocol to share live tables with other organizations or platforms without copying data. Recipients can read from Databricks, Spark, pandas or BI tools.

7. Delta Lake

Delta Lake is the default table format on Databricks: Parquet files plus a **transaction log** (**_delta_log/**) that records every change.

How the transaction log works

```
/prod/silver/orders/
├── _delta_log/
│   ├── 00000000000000000000.json     version 0: add files A, B
│   ├── 00000000000000000001.json     version 1: add file C
│   ├── 00000000000000000002.json     version 2: remove B, add D  (an UPDATE)
│   └── 00000000000000000010.checkpoint.parquet   periodic snapshot of the log
├── part-000A.snappy.parquet
├── part-000C.snappy.parquet
└── part-000D.snappy.parquet
```

- Each commit is a new JSON file listing files **added** and **removed**, plus metadata and stats (min/max per column for data skipping).
- Readers compute the current table from the log, so they never see half-written results — **atomic commits**.
- Concurrent writers use **optimistic concurrency**: if two commits touch the same files, one fails with a conflict and can retry.
- Updates and deletes don't modify files in place; they write new files and mark old ones removed (or use deletion vectors).

ACID on a data lake

Property	How Delta gives it
Atomicity	A commit file is written or not; partial writes are invisible
Consistency	Schema enforcement, constraints
Isolation	Snapshot isolation for reads; write conflict detection
Durability	Data and log on durable cloud storage

Creating and writing

```
CREATE TABLE prod.silver.orders (
  order_id BIGINT NOT NULL,
  customer_id BIGINT,
  amount DECIMAL(18,2),
  order_ts TIMESTAMP,
  order_date DATE GENERATED ALWAYS AS (CAST(order_ts AS DATE)),
  CONSTRAINT positive_amount CHECK (amount >= 0)
)
CLUSTER BY (order_date, customer_id)
COMMENT 'Cleaned orders, one row per order';
```

```
df.write.format("delta").mode("append").saveAsTable("prod.silver.orders")
(df.write.format("delta").mode("overwrite")
   .option("replaceWhere", "order_date = '2026-01-15'")
   .saveAsTable("prod.silver.orders"))
```

Schema enforcement and evolution

- **Enforcement:** writes with mismatched columns or types fail — bad data can't silently corrupt the table.
- **Evolution:** allow new columns explicitly.

```
df.write.format("delta").mode("append").option("mergeSchema", "true").saveAsTable("t")
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")   # for MERGE
```

```
ALTER TABLE t ADD COLUMNS (channel STRING);
ALTER TABLE t SET TBLPROPERTIES ('delta.columnMapping.mode' = 'name');  -- enables rename/drop column
ALTER TABLE t RENAME COLUMN status TO order_status;
```

DML

```
UPDATE prod.silver.orders SET status = 'CANCELLED' WHERE order_id = 1001;
DELETE FROM prod.silver.orders WHERE order_date < '2020-01-01';

MERGE INTO prod.silver.customers t
USING updates s ON t.customer_id = s.customer_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED AND s.op <> 'D' THEN INSERT *
WHEN NOT MATCHED BY SOURCE AND t.is_active THEN UPDATE SET is_active = false;
```

WHEN NOT MATCHED BY SOURCE handles target rows missing from the source — useful for full-snapshot syncs.

Time travel and restore

```
DESCRIBE HISTORY prod.silver.orders;                    -- versions, operations, who, when
SELECT * FROM prod.silver.orders VERSION AS OF 42;
SELECT * FROM prod.silver.orders TIMESTAMP AS OF '2026-01-15 06:00:00';
RESTORE TABLE prod.silver.orders TO VERSION AS OF 42;       -- undo a bad load
```

Time travel works only while old files still exist — **VACUUM** removes them after the retention period.

OPTIMIZE, Z-order and liquid clustering

```
OPTIMIZE prod.silver.orders;                                -- compact small files
OPTIMIZE prod.silver.orders WHERE order_date >= '2026-01-01'; -- partitioned tables: limit scope
OPTIMIZE prod.silver.orders ZORDER BY (customer_id);        -- co-locate values (legacy approach)
```

Liquid clustering is the current recommended layout for new tables. It replaces partitioning + Z-order for most cases.

```
CREATE TABLE t (...) CLUSTER BY (customer_id, order_date);
ALTER TABLE t CLUSTER BY (region);                    -- keys can change without rewriting all data
ALTER TABLE t CLUSTER BY AUTO;                        -- let Databricks pick keys from query patterns
OPTIMIZE t;                                           -- clusters incrementally
```

	Hive-style partitioning	Z-order	Liquid clustering
Layout	Folders per value	Re-sorts files on OPTIMIZE	Incremental clustering
Change keys	Rewrite table	Rerun on all data	**ALTER TABLE ... CLUSTER BY**
Small tables / high cardinality	Bad (tiny files)	OK	Good
Recommended for new tables	Only for very large tables with clear date filters	Legacy	Yes

VACUUM and retention

```
VACUUM prod.silver.orders;                            -- remove unreferenced files older than retention (default 7 days)
VACUUM prod.silver.orders RETAIN 168 HOURS;
ALTER TABLE t SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = '30 days',
  'delta.logRetentionDuration' = '60 days');
```

Shorter retention saves storage but limits time travel and can break long-running readers or streams that still need old files.

Deletion vectors

With deletion vectors enabled, **DELETE**, **UPDATE** and **MERGE** mark removed rows in a small side file instead of rewriting whole Parquet files. Writes are much faster; files are physically rewritten later by **OPTIMIZE**/purge.

Change Data Feed (CDF)

```
ALTER TABLE prod.silver.orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
SELECT * FROM table_changes('prod.silver.orders', 120);      -- changes since version 120
```

```
changes = (spark.read.format("delta").option("readChangeFeed", "true")
           .option("startingVersion", 120).table("prod.silver.orders"))
# _change_type: insert, update_preimage, update_postimage, delete
```

CDF lets downstream tables process only what changed instead of rescanning everything.

Sagar

Cloning

```
CREATE TABLE dev.silver.orders_copy SHALLOW CLONE prod.silver.orders;   -- metadata only, points to same files
CREATE TABLE backup.silver.orders_2026_01 DEEP CLONE prod.silver.orders; -- full independent copy
```

Shallow clones are perfect for testing a change on production-sized data without copying it.

Other useful features

- **Identity columns:** `id BIGINT GENERATED ALWAYS AS IDENTITY` for surrogate keys.
- **Generated columns:** derive `order_date` from `order_ts` automatically.
- **Constraints:** `NOT NULL`, `CHECK` (enforced); primary/foreign keys (informational).
- **UniForm:** writes Iceberg (and Hudi) metadata alongside Delta so Iceberg readers can read the same table.
- **Table properties:** `delta.autoOptimize.optimizeWrite`, `delta.autoOptimize.autoCompact`, `delta.appendOnly`, `delta.enableDeletionVectors`.

Sagar

8. Medallion architecture

The three layers

Layer	Content	Typical operations	Tables
Bronze	Raw data as ingested, append-only, plus metadata (**_ingest_ts**, **_source_file**)	Auto Loader / streaming append	Streaming tables, wide strings or raw JSON
Silver	Cleaned, typed, deduplicated, conformed entities	Parsing, casting, dedup, MERGE, SCD, quality checks	One table per business entity
Gold	Business-ready: facts, dimensions, aggregates, features	Joins, aggregations, KPIs	Star schemas, reporting tables, materialized views

Design principles

- Bronze keeps raw history so you can **reprocess** when logic changes or bugs are found.
- Silver is the **single source of truth** for each entity; downstream teams build on it.
- Gold is shaped for specific consumers (BI, ML, APIs) and can be rebuilt from silver.
- Each layer is idempotent and incremental where possible.
- Map layers to Unity Catalog: **prod.bronze.***, **prod.silver.***, **prod.gold.*** (or catalogs per layer/domain).

9. Ingestion

Auto Loader

Auto Loader incrementally ingests new files from cloud storage as they arrive, tracking which files were processed in the stream checkpoint.

```
(spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/Volumes/prod/meta/schemas/orders")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("abfss://raw@mylake.dfs.core.windows.net/orders/")
    .selectExpr("*", "_metadata.file_path AS _source_file", "current_timestamp() AS _ingest_ts")
 .writeStream
    .option("checkpointLocation", "/Volumes/prod/meta/checkpoints/orders_bronze")
    .option("mergeSchema", "true")
    .trigger(availableNow=True)
    .toTable("prod.bronze.orders"))
```

Key features:

- **Schema inference and evolution:** schema stored at **schemaLocation**; new columns handled by **schemaEvolutionMode** (**addNewColumns**, **rescue**, **failOnNewColumns**, **none**).
- **Rescued data column (_rescued_data):** values that don't fit the schema are kept here instead of being lost.
- **File discovery:** directory listing (default, simple) or file notification mode (cloud events + queue — scales to millions of files).
- **trigger(availableNow=True):** run as a scheduled batch job that processes only new files, then stops.

COPY INTO

```
COPY INTO prod.bronze.orders
FROM 'abfss://raw@mylake.dfs.core.windows.net/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Idempotent SQL load: files already loaded are skipped. Good for simpler, lower-volume batch loads; Auto Loader scales better.

Lakeflow Connect

Managed connectors that ingest from SaaS applications and databases (for example Salesforce, SQL Server, and others) into Unity Catalog tables with minimal code, including incremental/CDC ingestion for supported sources.

Other sources

- **Kafka / Azure Event Hubs:** Structured Streaming with the Kafka source (Event Hubs exposes a Kafka endpoint).
- **JDBC databases:** `spark.read.format("jdbc")` with parallel partitioning, or Lakeflow Connect, or ADF copying into the lake first.
- **Lakehouse Federation:** query external databases (SQL Server, PostgreSQL, Snowflake, etc.) through Unity Catalog without copying data — good for exploration and small lookups, not heavy ETL.

Sagar

10. Lakeflow Declarative Pipelines (formerly Delta Live Tables)

Declarative pipelines let you define **what** tables should contain; Databricks works out the dependency graph, runs it incrementally, manages checkpoints, retries, and enforces data-quality rules. The engine is now open-sourced as Apache Spark Declarative Pipelines.

Dataset types

Type	Behaviour	Use for
Streaming table	Processes each input row once, incrementally (append)	Bronze ingestion, append-only silver, CDC targets
Materialized view	Result kept up to date; recomputed incrementally where possible	Silver/gold transformations with joins and aggregations
View (temporary)	Not stored; intermediate logic inside the pipeline	Reusable steps

SQL example

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT *, _metadata.file_path AS source_file
FROM STREAM read_files('abfss://raw@mylake.dfs.core.windows.net/orders/', format => 'json');

CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id     EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_amount EXPECT (amount >= 0)          ON VIOLATION FAIL UPDATE,
  CONSTRAINT has_email    EXPECT (email IS NOT NULL)
)
AS SELECT CAST(order_id AS BIGINT) order_id, CAST(amount AS DECIMAL(18,2)) amount,
lower(email) email, order_ts
FROM STREAM(bronze_orders);

CREATE OR REFRESH MATERIALIZED VIEW gold_daily_sales
AS SELECT DATE(order_ts) order_date, SUM(amount) revenue, COUNT(*) orders
FROM silver_orders GROUP BY DATE(order_ts);
```

Sagar

Python example

```
from pyspark import pipelines as dp       # older pipelines: import dlt  (@dlt.table, @dlt.expect)
from pyspark.sql import functions as F

@dp.table(comment="Raw orders from landing")
def bronze_orders():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .load("abfss://raw@mylake.dfs.core.windows.net/orders/"))

@dp.table
@dp.expect_or_drop("valid_id", "order_id IS NOT NULL")
def silver_orders():
    return spark.readStream.table("bronze_orders").withColumn("amount",
F.col("amount").cast("decimal(18,2)"))
```

Expectations (data quality)

Clause	On violation
EXPECT (...)	Keep the row, record the metric
... ON VIOLATION DROP ROW	Drop the row, record it
... ON VIOLATION FAIL UPDATE	Stop the update

Expectation metrics appear in the pipeline UI and event log, so quality is measured continuously.

CDC and SCD with AUTO CDC (APPLY CHANGES)

```
CREATE OR REFRESH STREAMING TABLE dim_customer;

APPLY CHANGES INTO dim_customer          -- newer syntax: CREATE FLOW ... AS AUTO CDC
INTO ...
FROM STREAM(bronze_customer_cdc)
KEYS (customer_id)
APPLY AS DELETE WHEN op = 'D'
SEQUENCE BY op_ts                        -- orders out-of-order events correctly
COLUMNS * EXCEPT (op, op_ts)
STORED AS SCD TYPE 2;                    -- or SCD TYPE 1
```

This replaces a lot of hand-written MERGE logic: ordering by sequence column, deletes, and SCD2 history columns (**__START_AT**, **__END_AT**) are handled for you.

Running pipelines

- **Triggered** (process available data, then stop — scheduled by a job) or **continuous** (always on, low latency).
- **Development mode:** reuses the cluster, no automatic retries — fast iteration. **Production mode:** retries on failure, restarts clusters.

- *Full refresh rebuilds tables from scratch; normal updates are incremental.*
- *The **event log** records lineage, data-quality metrics, progress and errors — queryable as a table.*

Sagar

II. Lakeflow Jobs (orchestration)

Building blocks

Concept	Meaning
Job	A workflow with one or more tasks
Task	One unit: notebook, Python script, Python wheel, SQL file/query, dbt, pipeline, JAR, another job, If/else, For each
Dependencies	Tasks form a DAG; a task runs when its upstream tasks succeed (configurable: all succeeded, at least one, all done, etc.)
Compute	Job cluster (per job or shared across tasks), serverless, or SQL warehouse for SQL tasks
Parameters	Job parameters passed to every task; dynamic values like **{{job.start_time.iso_date}}**
Task values	Small values passed between tasks (**dbutils.jobs.taskValues**)

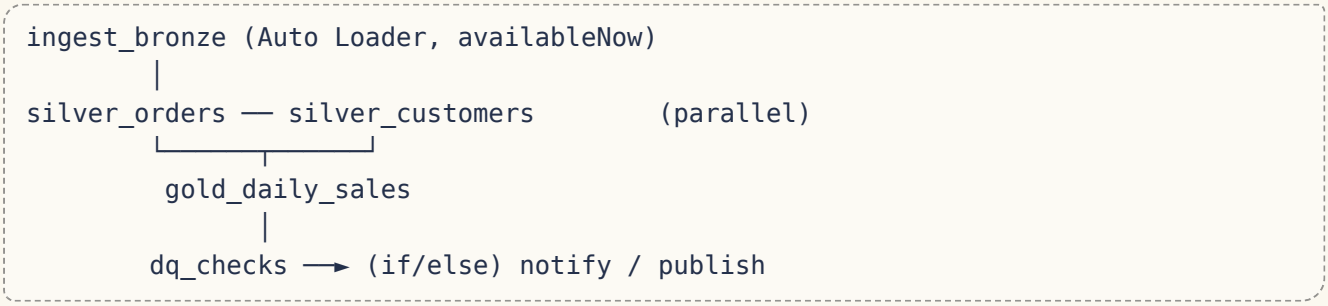
Triggers

- **Scheduled:** cron-like schedule with time zone.
- **File arrival:** run when new files land in a storage location or volume.
- **Table update:** run when upstream tables change.
- **Continuous:** restart immediately after each run (streaming jobs).
- **Manual / API:** from CI/CD, ADF or Airflow (**runs/submit**, **run-now**).

Reliability features

- **Retries** per task with delay; timeouts; max concurrent runs.
- **Repair run:** rerun only the failed tasks and their downstream tasks, not the whole job.
- **Notifications:** email, Slack/Teams webhooks, PagerDuty on start, success, failure, or duration threshold.
- **Run as** a service principal so production doesn't depend on a person's account.

A typical daily job



Orchestrating from outside

ADF has a native Databricks activity (notebook, Python, JAR, or run a job); Airflow has a Databricks provider (**DatabricksRunNowOperator**, **DatabricksSubmitRunOperator**). Use these

when the wider workflow spans other systems; keep Databricks-internal steps inside a Databricks job.

sagar

12. Databricks SQL

SQL warehouses

Type	Notes
Serverless	Starts in seconds, auto-scales, managed — the usual recommendation
Pro	Classic compute in your account with advanced features
Classic	Basic, in your account

Warehouses use Photon and are built for concurrency: many BI users and dashboards at once. Size (T-shirt sizes) controls per-query power; scaling (min/max clusters) controls concurrency.

What you build with it

- **SQL editor and queries** — parameterized, saved, scheduled.
- **Dashboards (AI/BI dashboards)** — visualizations over queries.
- **Alerts** — run a query on a schedule and notify when a condition is met (a freshness or data-quality alert, for example).
- **Connections from BI tools** — Power BI, Tableau, Looker via JDBC/ODBC or partner connectors.

Query profile

The query profile shows each operator's time, rows and bytes, spill, and whether Photon ran it — the SQL equivalent of the Spark UI. Use it to find scans that didn't prune, exploding joins and skew.

Sagar

13. Performance on Databricks

Engine-level features

Feature	What it does
Photon	Vectorized native execution for SQL/DataFrame operators
Adaptive Query Execution	Runtime join switching, partition coalescing, skew handling
Disk cache	Caches remote Parquet/Delta data on local SSDs of workers automatically
Data skipping	Min/max stats in the Delta log skip files that can't match a filter
Dynamic file pruning	Uses join results to skip files at runtime
Predictive optimization	Automatically runs OPTIMIZE, VACUUM and ANALYZE on UC managed tables

File layout

- Avoid many small files: enable optimized writes and auto-compaction, or rely on predictive optimization.
- Use liquid clustering on columns used in filters and joins.
- Don't over-partition: only partition very large tables (well above 1 TB) by a low-cardinality date column, if at all.
- Keep statistics fresh (`ANALYZE TABLE ... COMPUTE STATISTICS`) when not using predictive optimization.

Spark-level habits (still apply)

- Broadcast small dimensions; fix skew; prune columns and filter early.
- Avoid Python UDFs; use built-in functions or pandas UDFs.
- Right-size shuffle partitions (or let AQE handle it with `spark.sql.shuffle.partitions=auto` on Databricks).

Choosing cluster size

- Start small and scale based on the Spark UI: if CPU is saturated and tasks queue, add workers; if there's heavy spill, choose memory-optimized nodes or more partitions.
- Jobs with big shuffles benefit from storage-optimized nodes (fast local SSD).
- For many small jobs, use serverless or a pool to cut startup time.

14. Security

Identities

- **Users** and **groups** synced from Microsoft Entra ID (Azure AD) via SCIM.
- **Service principals** for automation — jobs, CI/CD, external tools. Use OAuth (machine-to-machine) instead of personal access tokens where possible.
- Workspace roles (admin, user) and entitlements (cluster creation, SQL access).

Secrets

```
databricks secrets create-scope kv-prod --scope-backend-type AZURE_KEYVAULT \
  --resource-id <key-vault-resource-id> --dns-name https://<vault>.vault.azure.net/
```

- **Azure Key Vault-backed scopes:** secrets live in Key Vault; Databricks reads them.
- **Databricks-backed scopes:** stored by Databricks.
- Grant **READ** on scopes to groups/service principals. Values are redacted if printed in notebooks — but never try to print them.

Network security (Azure)

- **VNet injection:** deploy classic compute into your own VNet with your NSG rules and routing.
- **Secure cluster connectivity (no public IP):** cluster nodes have no public IPs.
- **Private Link:** private connectivity for users and compute to the control plane, and to storage.
- **IP access lists** for workspace access.

Data protection

- Unity Catalog permissions, row filters and column masks.
- Encryption at rest (cloud-managed or customer-managed keys) and in transit.
- Audit logs in system tables.

15. CI/CD and DevOps

Git folders

Clone repos into the workspace, branch, commit and pull directly from the UI. Production should run code from a specific branch/tag deployed by CI, not from someone's personal folder.

Databricks Asset Bundles (DABs)

Bundles define jobs, pipelines, clusters and other resources as YAML alongside your code, and deploy them per environment.

```yaml
# databricks.yml
bundle:
  name: orders_pipeline

targets:
  dev:
    mode: development
    workspace: { host: https://adb-111.azuredatabricks.net }
  prod:
    mode: production
    workspace: { host: https://adb-222.azuredatabricks.net }
    run_as: { service_principal_name: "etl-sp" }

resources:
  jobs:
    orders_daily:
      name: orders_daily
      schedule: { quartz_cron_expression: "0 0 2 * * ?", timezone_id: "Asia/Kolkata" }
      tasks:
        - task_key: ingest
          notebook_task: { notebook_path: ./src/ingest.py }
        - task_key: transform
          depends_on: [{ task_key: ingest }]
          python_wheel_task: { package_name: orders_pipeline, entry_point: main }
```

```
databricks bundle validate
databricks bundle deploy -t dev
databricks bundle run orders_daily -t dev
databricks bundle deploy -t prod          # from CI after tests pass
```

Typical pipeline

1. Developer works in a feature branch, deploys to dev with **bundle deploy -t dev**.
2. Pull request triggers CI: lint, unit tests (local Spark), **bundle validate**.
3. Merge to main deploys to test; integration tests run against test data.
4. Approved release deploys to prod with a service principal.

Tools: Azure DevOps Pipelines or GitHub Actions running the Databricks CLI; Terraform for workspaces, Unity Catalog objects, clusters and permissions.

Sagar

Testing on Databricks

- Unit-test transformation functions with a local SparkSession (pytest) — fast and cheap.
- Integration tests on a small dataset in a test catalog.
- Data-quality expectations in pipelines; shallow clones of prod tables for realistic tests.

Sagar

16. Cost management

How billing works

You pay **DBUs** (Databricks Units, per second of compute, rate depends on compute type and tier) plus the **cloud VM and storage cost** for classic compute. Serverless includes the infrastructure in its DBU price.

Cost levers

Lever	Effect
Job compute instead of all-purpose	Lower DBU rate for scheduled work
Auto-termination	Stops paying for idle clusters
Autoscaling with sensible max	Avoids over-provisioning
Spot workers	Cheaper VMs for fault-tolerant jobs
Serverless	No idle time, no start-up waste for short jobs
Photon	Higher rate but often fewer total DBUs for SQL-heavy work — measure
Right-sized SQL warehouses + auto-stop	BI cost control
Cluster policies + tags	Enforce limits, attribute cost to teams
Efficient code and layout	Fewer DBUs: less data scanned and shuffled

Monitoring spend

```
SELECT usage_date, sku_name, SUM(usage_quantity) AS dbus
FROM system.billing.usage
WHERE usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY usage_date, sku_name
ORDER BY usage_date;
```

Join billing usage with job metadata and tags to see cost per job, per team, per environment.

17. Monitoring and troubleshooting

Where to look

Tool	Use
Job run page	Task status, durations, error messages, repair runs
Spark UI	Stages, tasks, skew, spill, plans
Driver logs (stdout/stderr/log4j)	Python exceptions, stack traces
Cluster event log	Start, resize, termination, spot loss
Pipeline event log	Declarative pipeline progress, expectations, lineage
Query profile / query history	SQL warehouse performance
System tables	Job runs, billing, audit, lineage across the account

Common problems

Problem	Typical cause	Fix
ConcurrentAppendException / conflict errors	Two writes touched the same files (e.g. MERGE + OPTIMIZE, or parallel MERGEs)	Make conditions partition-specific, serialize writers, retry
MERGE fails: multiple source rows matched	Duplicate keys in source	Deduplicate the source first
Job much slower than usual	Data growth, skew, small files, cluster change	Compare runs in Spark UI, OPTIMIZE, fix skew
Driver OOM	**collect**/**toPandas** on large data, big broadcast	Write instead of collect, avoid broadcasting large tables
Stream restarted from the beginning	Checkpoint deleted or changed	Keep checkpoints stable, one per stream
FileNotFoundException reading Delta	Files vacuumed while a long reader needed them	Increase retention, avoid VACUUM with short retention
Permission denied	Missing UC grant (**USE CATALOG**/**USE SCHEMA**/**SELECT**) or external location access	Check **SHOW GRANTS**, grant to the right group/SP
Schema mismatch on write	New/changed columns	Fix upstream, or allow **mergeSchema** deliberately

18. End-to-end example

A daily orders pipeline on Azure Databricks with Unity Catalog.

1. Bronze: Auto Loader ingestion

```
(spark.readStream.format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", "/Volumes/prod/meta/schemas/orders")
   .load("abfss://raw@mylake.dfs.core.windows.net/orders/")
   .select("*", F.col("_metadata.file_path").alias("_source_file"),
F.current_timestamp().alias("_ingest_ts"))
 .writeStream
   .option("checkpointLocation", "/Volumes/prod/meta/checkpoints/orders_bronze")
   .trigger(availableNow=True)
   .toTable("prod.bronze.orders"))
```

2. Silver: clean, dedup, MERGE

```
from delta.tables import DeltaTable
from pyspark.sql import functions as F, Window as W

def upsert_silver(batch_df, batch_id):
    clean = (batch_df
      .withColumn("order_id", F.col("order_id").cast("bigint"))
      .withColumn("amount", F.col("amount").cast("decimal(18,2)"))
      .withColumn("order_ts", F.to_timestamp("order_ts"))
      .filter("order_id IS NOT NULL")
      .withColumn("rn",
F.row_number().over(W.partitionBy("order_id").orderBy(F.desc("updated_at"))))
      .filter("rn = 1").drop("rn"))
    (DeltaTable.forName(spark, "prod.silver.orders").alias("t")
       .merge(clean.alias("s"), "t.order_id = s.order_id")
       .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
       .whenNotMatchedInsertAll()
       .execute())

(spark.readStream.table("prod.bronze.orders")
   .writeStream.foreachBatch(upsert_silver)
   .option("checkpointLocation", "/Volumes/prod/meta/checkpoints/orders_silver")
   .trigger(availableNow=True)
   .start())
```

3. Gold: business table

```
CREATE OR REPLACE TABLE prod.gold.daily_sales
CLUSTER BY (order_date)
AS SELECT CAST(order_ts AS DATE) AS order_date, c.region,
          COUNT(*) AS orders, SUM(o.amount) AS revenue
FROM prod.silver.orders o
JOIN prod.silver.customers c USING (customer_id)
GROUP BY ALL;
```

4. Quality gate

```
SELECT COUNT(*) AS dupes FROM (
  SELECT order_id FROM prod.silver.orders GROUP BY order_id HAVING COUNT(*) > 1);
-- task fails (or alerts) if dupes > 0
```

5. Orchestrate

One Lakeflow Job: **bronze** → **silver** → **gold** → **dq_check**, job compute or serverless, scheduled at 02:00 IST, retries on each task, failure notification to the on-call channel, running as a service principal, deployed with an asset bundle.

> On Databricks the platform does a lot for you — but the fundamentals stay the same: idempotent writes, incremental processing, good file layout, clear ownership of each layer, and permissions granted to groups and service principals.

— **Sagar**